IE 5311-001 (D01) Principles of Optimization

Lidivine Kengne

Homework 05

Problem 1. The primal simplex method [50 pts]

Throughout, the LP is

$$\max\ c^\top x \qquad \text{s.t.} \quad Ax \le b, \quad x \ge 0,$$

with $A \in \mathbb{R}_+^{m\times n}$, $b \in \mathbb{R}_+^m$, $c \in \mathbb{R}_+^n$, $b > 0$ and the feasible region bounded. Adding one slack per row gives the standard form

$$\max\ \bar{c}^\top \bar{x} \qquad \text{s.t.} \quad T\bar{x} = b, \quad \bar{x} \ge 0, \qquad T = \begin{bmatrix} A & I \end{bmatrix}, \quad \bar{c} = \begin{pmatrix} c \\ 0 \end{pmatrix}.$$

Because $b > 0$, the slack basis is feasible and no Phase I is needed.

At a basis B with nonbasis N, writing A_B for the basis matrix:

$$x_B = A_B^{-1} b, \qquad x_N = 0, \qquad z = \bar{c}_B^\top x_B,$$

$$y^\top = \bar{c}_B^\top A_B^{-1} \quad (\text{solve } A_B^\top y = \bar{c}_B), \qquad \bar{c}_j = \bar{c}_j^{\text{orig}} - y^\top T_j \quad \text{for } j \in N.$$

For a maximization problem a nonbasic j is attractive when $\bar{c}_j > 0$; optimality holds when $\bar{c}_j \le 0$ for every $j \in N$. With entering column q, the direction is $d = A_B^{-1} T_q$ and the ratio test is

$$\theta = \min_{i\,:\,d_i > 0} \frac{(x_B)_i}{d_i},$$

the minimizing row giving the leaving variable.

1. The simplex method by hand

$$\begin{array}{ll} \max & 3x_1 + 2x_2 + x_3 \\ \text{s.t.} & x_1 \le 4, \\ & x_1 + x_2 \le 8, \\ & x_1 + x_2 + x_3 \le 12, \\ & x_1, x_2, x_3 \ge 0. \end{array}$$

Introduce slacks $x_4, x_5, x_6 \ge 0$:

$$\begin{array}{lcl} x_1 + x_4 = 4 \\ x_1 + x_2 + x_5 = 8 \\ x_1 + x_2 + x_3 + x_6 = 12 \end{array} \qquad T = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}, \quad b = \begin{pmatrix} 4 \\ 8 \\ 12 \end{pmatrix}.$$

Iteration 0

(a) $B = \{x_4, x_5, x_6\}$, $N = \{x_1, x_2, x_3\}$, and $A_B = I_3$.

(b) $x_B = A_B^{-1} b = (4, 8, 12)$, $x_N = (0, 0, 0)$, objective $z = 0$.

(c) $\bar{c}_B = 0$, so $y = 0$ and the reduced costs are the original costs:

$$\bar{c}_1 = 3, \qquad \bar{c}_2 = 2, \qquad \bar{c}_3 = 1.$$

All are positive; the largest is $\bar{c}_1 = 3$, so x_1 **enters**.

(d) $d = A_B^{-1} T_1 = (1, 1, 1)$. Ratios $\frac{4}{1} = 4$, $\frac{8}{1} = 8$, $\frac{12}{1} = 12$; the minimum is 4 in row 1, so x_4 **leaves** with step $\theta = 4$.

(e) $x_1 = 4$, $x_5 = 8 - 4 = 4$, $x_6 = 12 - 4 = 8$, all others zero; $z = 12$.

Iteration 1

(a) $B = \{x_1, x_5, x_6\}$, $N = \{x_2, x_3, x_4\}$,

$$A_B = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}.$$

(b) Solving $A_B x_B = b$ by forward substitution gives $x_1 = 4$, then $x_5 = 8 - 4 = 4$ and $x_6 = 12 - 4 = 8$: $x_B = (4, 4, 8)$, $x_N = 0$, $z = 3(4) = 12$.

(c) $\bar{c}_B = (3, 0, 0)$. Solving $A_B^\top y = \bar{c}_B$, that is $y_2 = 0$, $y_3 = 0$, $y_1 + y_2 + y_3 = 3$, gives $y = (3, 0, 0)$. Then

$$\bar{c}_2 = 2 - y^\top (0, 1, 1) = 2, \qquad \bar{c}_3 = 1 - y^\top (0, 0, 1) = 1, \qquad \bar{c}_4 = 0 - y^\top (1, 0, 0) = -3.$$

x_2 **enters** ($\bar{c}_2 = 2$ is largest).

(d) $d = A_B^{-1} T_2$ solves $A_B d = (0, 1, 1)$, giving $d = (0, 1, 1)$. Only rows 2 and 3 have $d_i > 0$, with ratios $\frac{4}{1} = 4$ and $\frac{8}{1} = 8$. The minimum is 4, so x_5 **leaves**, $\theta = 4$.

(e) $x_1 = 4$, $x_2 = 4$, $x_6 = 8 - 4 = 4$; $z = 20$.

Iteration 2

(a) $B = \{x_1, x_2, x_6\}$, $N = \{x_3, x_4, x_5\}$,

$$A_B = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}.$$

(b) $x_1 = 4$, $x_2 = 8 - 4 = 4$, $x_6 = 12 - 4 - 4 = 4$, so $x_B = (4, 4, 4)$ and $z = 3(4) + 2(4) = 20$.

(c) $\bar{c}_B = (3, 2, 0)$. Solving $A_B^\top y = \bar{c}_B$, that is $y_3 = 0$, $y_2 + y_3 = 2$, $y_1 + y_2 + y_3 = 3$, gives $y = (1, 2, 0)$. Then

$$\bar{c}_3 = 1 - y^\top (0, 0, 1) = 1, \qquad \bar{c}_4 = 0 - y^\top (1, 0, 0) = -1, \qquad \bar{c}_5 = 0 - y^\top (0, 1, 0) = -2.$$

x_3 **enters**, the only positive reduced cost.

(d) $A_B d = (0,0,1)$ gives $d = (0,0,1)$. Only row 3 has $d_i > 0$, ratio $\frac{4}{1} = 4$, so x_6 **leaves**, $\theta = 4$.

(e) $x_1 = x_2 = x_3 = 4$; $z = 24$.

Iteration 3: optimality

(a) $B = \{x_1, x_2, x_3\}$, $N = \{x_4, x_5, x_6\}$, with the same A_B as above.

(b) $x_B = (4,4,4)$, $z = 3(4) + 2(4) + 1(4) = 24$.

(c) $\bar{c}_B = (3,2,1)$ gives $y_3 = 1$, $y_2 = 1$, $y_1 = 1$, so $y = (1,1,1)$ and

$$\bar{c}_4 = -1, \qquad \bar{c}_5 = -1, \qquad \bar{c}_6 = -1.$$

Every reduced cost is non-positive, so the **optimality condition is satisfied** and the method stops after three pivots, as the hint predicts.

$$\boxed{x^\star = (4,4,4), \qquad z^\star = 24, \qquad y^\star = (1,1,1).}$$

All three constraints are tight at $x^\star$, and the dual vector $y^\star = (1,1,1)$ gives the shadow price of each resource. Dual feasibility $A^\top y^\star = (3,2,1) \geq c$ and $b^\top y^\star = 24 = c^\top x^\star$ confirm optimality by strong duality, independently of the pivoting.

The arithmetic, carried out in Excel. Part 1 permits a calculator or a spreadsheet, and the workbook `p1_simplex_tableau.xlsx` carries the computation. It runs the equivalent *tableau* form of the same three pivots: the four tableaux sit one under another, and each is produced from the one above by row operations written as live formulas,

$$\text{new pivot row} = \frac{\text{old pivot row}}{\text{pivot element}},$$
$$\text{every other row} = \text{old row} - (\text{its pivot-column entry}) \times \text{new pivot row},$$

applied to the constraint rows and to the $\bar{c}$ row alike. The ratio column is an `IF` that divides the right-hand side by the pivot-column entry only where that entry is positive, which is why it is blank on exactly the rows the ratio test skips. Nothing in the sheet is a typed-in result except the initial tableau.

The two forms agree at every step: bases $\{x_4, x_5, x_6\} \to \{x_1, x_5, x_6\} \to \{x_1, x_2, x_6\} \to \{x_1, x_2, x_3\}$, the same reduced-cost rows $(3,2,1,0,0,0)$, $(0,2,1,-3,0,0)$, $(0,0,1,-1,-2,0)$, $(0,0,0,-1,-1,-1)$, and objectives $0, 12, 20, 24$. In the final tableau the entries under the slack columns x_4, x_5, x_6 are $-1,-1,-1$, so the duals can be read straight off as $y^\star = (1,1,1)$ without solving a separate system — one practical advantage of the tableau arrangement over the matrix form used above.

2. Implementation

`p1_simplex.py` provides `simplex(A, b, c)`, which returns the optimal x, the objective, the iteration count and a status flag. It is a *revised* simplex: rather than carrying a full tableau it stores only B and N and, each iteration, solves

$$A_B x_B = b, \qquad A_B^\top y = \bar{c}_B, \qquad A_B d = T_q,$$

computes the reduced costs $\bar{c}_j = \bar{c}_j - y^\top T_j$, enters the most positive one (Dantzig's rule, lowest index on ties), and leaves by the ratio test. It reports `unbounded` when $d \le 0$.

Run with `log=True` it prints, at each iteration, exactly the items (a)–(e) asked for in Part 1. Its output reproduces the hand computation above line for line: the same bases $\{x_4, x_5, x_6\} \to \{x_1, x_5, x_6\} \to \{x_1, x_2, x_6\} \to \{x_1, x_2, x_3\}$, the same entering variables x_1, x_2, x_3, the same leaving variables x_4, x_5, x_6, the same step size $\theta = 4$ at every pivot, the objective sequence $0 \to 12 \to 20 \to 24$, and the same optimum $x^\star = (4, 4, 4)$ with $z^\star = 24$.

3. Random resource allocation instances

`p1_benchmark.py` generates instances of the stated form: producing one unit of product i earns c_i and consumes a_{ji} units of resource j, of capacity b_j. With

$$c_i \sim U(1, 10), \qquad a_{ji} \sim U(0.5, 5), \qquad b_j \sim U(0.3, 0.6) \cdot \sum_i a_{ji},$$

and seed 5311 fixed for reproducibility. Two properties are deliberate. Every $a_{ji} > 0$, so each column of A has a positive entry and the feasible region is bounded, as Problem 1 assumes. Every $b_j > 0$, so the slack basis is feasible and the Phase I the implementation lacks is not needed. Drawing b_j as a fraction of the row sum keeps several capacity rows tight at the optimum rather than making them all slack.

Five sizes are generated: $(n, m) = (5, 3)$, $(10, 5)$, $(20, 10)$, $(50, 20)$, $(100, 40)$.

4. Comparison against Gurobi

Gurobi was run with `Method=0` (primal simplex) and `Presolve=0` so that its iteration count measures the same thing as the implementation's. Timings are in milliseconds and cover the solve only, not model construction.

n	m	obj. mine	obj. Gurobi	rel. gap	it. mine	it. grb	t mine	t grb	ratio†
5	3	13.068567	13.068567	0	1	1	0.076	0.035	3.4×
10	5	33.979881	33.979881	0	3	5	0.288	0.050	6.9×
20	10	63.544405	63.544405	2.2×10^{-16}	9	10	1.145	0.104	14.1×
50	20	166.285445	166.285445	1.7×10^{-16}	20	25	4.469	0.219	17.1×
100	40	371.508091	371.508091	0	31	26	13.344	0.512	23.5×

Objective values. Identical on all five instances; the largest relative gap is 2.2×10^{-16}, which is one unit in the last place of a double. The implementation is correct on these instances.

Solutions. The largest componentwise difference $\max_i |x_i^{\text{mine}} - x_i^{\text{Gurobi}}|$ runs from 0 to 9.8×10^{-15}. The two solvers return the *same vertex*, not merely the same objective. That is not guaranteed in general: when an LP has alternative optima the two can stop at different vertices with equal objective, and a correct implementation would still disagree componentwise. On these random instances the optimum is unique, so the question does not arise.

Iteration counts. The same order of magnitude throughout: $1, 3, 9, 20, 31$ against Gurobi's $1, 5, 10, 25, 26$. Both grow roughly like m, the number of rows, matching the usual empirical observation that the simplex method takes a small multiple of m pivots. The differences come from the pricing rule. Dantzig's rule, used here, enters the largest reduced cost, which is scale dependent and ignores how far the step will actually go. Gurobi defaults to a steepest-edge or devex rule, which normalizes the reduced cost by the length of the resulting edge and typically buys more objective per pivot. That is why Gurobi needs fewer pivots on the $(50, 20)$ instance and why it stays flat at 26 on the largest one while the implementation climbs to 31.

Runtimes. Timings at this scale are not reproducible to their digits, so they are reported as follows. Each tabulated time is the median of seven solves from one representative run; the ratio column marked † is instead the median over *five independent repetitions of the whole benchmark*, with the observed range across those repetitions given below.

n	m	slowest ratio	median ratio	fastest ratio
5	3	4.6×	3.4×	2.8×
10	5	8.9×	6.9×	4.5×
20	10	14.4×	14.1×	12.0×
50	20	18.1×	17.1×	15.7×
100	40	25.1×	23.5×	22.1×

The implementation is slower throughout, and the ratio *increased monotonically with m in every one of the five repetitions*, rising from roughly 3× at $m = 3$ to roughly 24× at $m = 40$. That ordering, not the individual figures, is the result. Per-iteration cost explains it. Each iteration here performs three dense solves with A_B *from scratch*, at $O(m^3)$ each. A production simplex factorizes A_B once and then *updates* the factorization after a pivot at $O(m^2)$, refactorizing only every few dozen iterations, and it stores A sparsely so that an $m \times m$ basis with few nonzeros never becomes a dense m^2 array. Gurobi is also compiled rather than interpreted. The gap is therefore structural and widens with m; it is not a matter of constant factors.

Remark 1. Both solvers run in well under a millisecond on the smaller instances, so the absolute times sit close to the resolution of the timer and the overhead of the Python call. Taking the median of seven repetitions removes most of that, but the comparison is still a trend over a factor of 13 in m rather than a precise measurement of either solver. The direction of the trend is what the complexity argument predicts.

5. Limitations on general LPs

`p1_limitations.py` demonstrates the first two of these rather than asserting them.

No Phase I, and the failure is silent. The method starts from the slack basis, which is feasible only when $b \geq 0$. Given

$$\max\ x_1 \quad \text{s.t.} \quad x_1 \leq 1,\ \ -x_1 \leq -3,\ \ x_1 \geq 0$$

— infeasible, since $x_1 \leq 1$ and $x_1 \geq 3$ cannot both hold — the implementation returns status `optimal` with $x_1 = 1$ and objective 1. The returned point violates the second row by 2. Gurobi

reports `INFEASIBLE`. Nothing in the code detects this: a Phase I with artificial variables is needed both to find a starting basis and to certify infeasibility.

Cycling. Dantzig's rule carries no anti-cycling guarantee. On Beale's classic degenerate example

$$\max\ 0.75x_1 - 150x_2 + 0.02x_3 - 6x_4$$

subject to $0.25x_1 - 60x_2 - 0.04x_3 + 9x_4 \le 0$, $0.5x_1 - 90x_2 - 0.02x_3 + 3x_4 \le 0$, $x_3 \le 1$, $x \ge 0$, the implementation *cycles* and hits its iteration cap without terminating. Bland's rule (lowest index among all attractive columns) or a lexicographic ratio test would guarantee finite termination, at the cost of more pivots in the typical case.

Restricted form. Only $\max c^\top x$ subject to $Ax \le b$, $x \ge 0$ is accepted. Equality rows, $\ge$ rows, free variables and general bounds $\ell \le x \le u$ must be converted by hand first, and every conversion adds variables or rows. A production code handles bounds natively, keeping the basis size at m rather than inflating it.

Dense linear algebra. Discussed in Part 4: $O(m^3)$ per iteration instead of $O(m^2)$, and a dense $m \times m$ basis even when A is sparse. For the large sparse LPs where the simplex method is actually used this is disqualifying.

Numerical fragility. One fixed tolerance 10^{-9} is used for every comparison, regardless of how the problem is scaled. There is no scaling, no Harris ratio test, and no basis-condition monitoring or repair. An ill-conditioned A_B can produce a wrong pivot with no warning.

No presolve, warm start or dual simplex. Redundant rows, fixed variables and singleton rows are never removed. There is no way to re-solve from a previous basis after a bound change, which is exactly what branch and bound needs; this alone makes the code unusable as an integer-programming subroutine.

Problem 2. GAP, Dantzig–Wolfe and column generation [70 pts]

Sets and parameters

Symbol	Meaning
$I = [n]$	jobs, indexed by i
$J = [m]$	agents, indexed by j
c_{ij}	cost of assigning job i to agent j
a_{ij}	resource of agent j consumed by job i
b_j	resource capacity of agent j

1. GAP as a binary integer program

With $x_{ij} \in \{0, 1\}$ equal to one when job i is assigned to agent j:

$$\min \quad \sum_{i \in I} \sum_{j \in J} c_{ij}\, x_{ij} \tag{1}$$

$$\text{s.t.} \quad \sum_{j \in J} x_{ij} = 1 \qquad \forall\, i \in I \tag{2}$$

$$\sum_{i \in I} a_{ij}\, x_{ij} \le b_j \qquad \forall\, j \in J \tag{3}$$

$$x_{ij} \in \{0, 1\} \qquad \forall\, i \in I,\ j \in J. \tag{4}$$

Row (2) sends each job to exactly one agent; row (3) is agent j's capacity.

Classification (slides 8–10). Constrained; nm binary variables, compact; linear objective; $n+m$ linear rows, compact; single objective; deterministic; one stage; one player.

2. Dantzig–Wolfe reformulation

The capacity rows (3) do not link agents: row j involves only the column $x_{\cdot j} = (x_{1j}, \ldots, x_{nj})$. Only (2) couples the agents. That is exactly the structure Dantzig–Wolfe exploits: keep (2) in the master and push (3) into a subproblem per agent.

Feasible assignment patterns. For each agent j define

$$\boxed{X_j \;:=\; \Big\{ x \in \{0,1\}^n \;:\; \sum_{i \in I} a_{ij}\, x_i \le b_j \Big\}}$$

the set of job subsets that fit within agent j's capacity. X_j is a knapsack feasible set: finite, nonempty (it contains $x = 0$), and bounded, so $\mathrm{conv}(X_j)$ is a polytope with *no extreme rays* and every point of it is a convex combination of the elements of X_j. Enumerate $X_j = \{x^{jk}\}_{k \in K_j}$.

Substitution. Writing each agent's column as a convex combination of its patterns,

$$x_{ij} \;=\; \sum_{k\in K_j} x_i^{jk}\,\lambda_{jk}, \qquad \sum_{k\in K_j}\lambda_{jk}=1, \qquad \lambda_{jk}\ge 0,$$

and defining the pattern cost

$$C_{jk} \;:=\; \sum_{i\in I} c_{ij}\,x_i^{jk},$$

substitution into (1)–(2) gives the **Dantzig–Wolfe master**:

$$\min_{\lambda} \quad \sum_{j\in J}\sum_{k\in K_j} C_{jk}\,\lambda_{jk} \tag{5}$$

$$\text{s.t.} \quad \sum_{j\in J}\sum_{k\in K_j} x_i^{jk}\,\lambda_{jk}=1 \qquad \forall\, i\in I \qquad \text{(each job covered once)} \tag{6}$$

$$\sum_{k\in K_j}\lambda_{jk}=1 \qquad \forall\, j\in J \qquad \text{(convexity, one per agent)} \tag{7}$$

$$\lambda_{jk}\in\{0,1\}. \tag{8}$$

The capacity rows have disappeared: they are enforced implicitly, because every generated column is an element of some X_j and therefore already respects capacity. The master has $n+m$ rows and $\sum_j |K_j|$ columns, which is exponential in n — hence column generation.

3. Identical agents

Now $a_{ij}=a_i$ and $b_j=b$ for all i,j, while c_{ij} may still differ.

The pattern set no longer depends on j:

$$\boxed{X_j = X := \Big\{x\in\{0,1\}^n : \sum_{i\in I} a_i x_i \le b\Big\} \quad \text{for every } j\in J.}$$

One shared pool of columns $X=\{x^k\}_{k\in K}$ serves all m agents, so the master becomes

$$\min_{\lambda} \quad \sum_{j\in J}\sum_{k\in K}\Big(\sum_{i\in I} c_{ij}x_i^k\Big)\lambda_{jk} \tag{9}$$

$$\text{s.t.} \quad \sum_{j\in J}\sum_{k\in K} x_i^k\,\lambda_{jk}=1 \qquad \forall\, i\in I \tag{10}$$

$$\sum_{k\in K}\lambda_{jk}=1 \qquad \forall\, j\in J, \qquad \lambda\ge 0. \tag{11}$$

Three concrete savings:

- **Storage.** One pattern pool of size $|X|$ instead of m pools totalling $m|X|$. A pattern generated for one agent is immediately available to every other.
- **Pricing.** The subproblems share a single feasible set. Only the objective changes with j, so pricing is a family of knapsacks over one common constraint $\sum_i a_i x_i \le b$ rather than m unrelated problems, and any preprocessing of that constraint (sorting by a_i, dominance, a dynamic-programming table over capacities $0,\dots,b$) is done once and reused for all m agents.

- **Symmetry.** The m convexity rows are interchangeable in their constraint structure and differ only through the costs.

Remark 2. If the costs were identical as well, $c_{ij} = c_i$, the agents would be fully interchangeable and one could aggregate further: replace λ_{jk} by $\nu_k := \sum_j \lambda_{jk}$, the *number* of agents using pattern k, collapsing the m convexity rows (7) into the single row $\sum_{k\in K} \nu_k = m$ with ν nonnegative and integer. That removes the symmetry that otherwise makes branch and bound explore $m!$ equivalent solutions. Since the problem states that the c_{ij} may differ, that last step is not available here and the m convexity rows must stay.

From here on, the integrality constraints are relaxed: $\lambda \geq 0$.

4. Restricted master problem

Column generation never builds all of K. Let $K' \subseteq K$ be the columns generated so far, with K'_j those available to agent j. The **restricted master problem** (RMP) is the master over K' only:

$$\text{(RMP)} \qquad \min_{\lambda} \quad \sum_{j\in J} \sum_{k\in K'_j} C_{jk}\, \lambda_{jk} \tag{12}$$

$$\text{s.t.} \quad \sum_{j\in J} \sum_{k\in K'_j} x_i^k\, \lambda_{jk} = 1 \qquad \forall\, i \in I \qquad [\pi_i] \tag{13}$$

$$\sum_{k\in K'_j} \lambda_{jk} = 1 \qquad \forall\, j \in J \qquad [\mu_j] \tag{14}$$

$$\lambda_{jk} \geq 0. \tag{15}$$

The bracketed symbols are the dual variables attached to each row; both families are *free*, the rows being equalities. Since the RMP optimizes over a subset of the master's columns, its optimal value is an **upper bound** on the master LP value.

5. Lagrangian, dual, and the reduced cost of a column

Lagrangian. Dualize both row families with multipliers $\pi \in \mathbb{R}^n$ and $\mu \in \mathbb{R}^m$:

$$L(\lambda, \pi, \mu) = \sum_j \sum_k C_{jk}\lambda_{jk} + \sum_{i\in I} \pi_i \Big(1 - \sum_j \sum_k x_i^k \lambda_{jk}\Big) + \sum_{j\in J} \mu_j \Big(1 - \sum_k \lambda_{jk}\Big).$$

Collecting the λ_{jk} terms,

$$\boxed{L(\lambda, \pi, \mu) = \sum_{i\in I} \pi_i + \sum_{j\in J} \mu_j + \sum_{j\in J} \sum_{k\in K_j} \Big[C_{jk} - \sum_{i\in I} \pi_i x_i^k - \mu_j\Big] \lambda_{jk}.}$$

Dual of the master LP. Minimizing L over $\lambda \geq 0$ is $-\infty$ unless every bracket is nonnegative, which gives the dual

$$\boxed{\begin{aligned} \max_{\pi,\mu} \quad & \sum_{i\in I} \pi_i + \sum_{j\in J} \mu_j \\ \text{s.t.} \quad & \sum_{i\in I} \pi_i x_i^k + \mu_j \;\leq\; C_{jk} \quad \forall\, j \in J,\ k \in K_j \\ & \pi \in \mathbb{R}^n,\ \mu \in \mathbb{R}^m \text{ free.} \end{aligned}}$$

One dual row per *column* of the master: the dual has exponentially many constraints, which is the mirror image of the master having exponentially many variables, and is why pricing is a separation problem in disguise.

Reduced cost. The bracket in L is exactly the reduced cost of column (j, k):

$$\boxed{\bar{c}_{jk} = C_{jk} - \sum_{i \in I} \pi_i x_i^k - \mu_j = \sum_{i \in I} \big(c_{ij} - \pi_i\big) x_i^k \;-\; \mu_j.}$$

Read it as: using pattern k at agent j costs c_{ij} per job taken, against a credit π_i for relieving job i's covering row, less the fixed charge μ_j for occupying agent j's single convexity slot.

6. The pricing subproblem

A column improves the RMP only if its reduced cost is negative. We therefore minimize $\bar{c}_{jk}$ over *all* patterns, which, since μ_j is a constant for fixed j, decomposes by agent:

$$\boxed{z_j(\pi, \mu) \;=\; \Big[\min_{x \in \{0,1\}^n} \sum_{i \in I} \big(c_{ij} - \pi_i\big) x_i \quad \text{s.t.} \quad \sum_{i \in I} a_i x_i \le b \Big] \;-\; \mu_j.}$$

Equivalently, substituting $\max(-\,\cdot\,) = -\min(\cdot)$, it is the **0–1 knapsack problem**

$$\max_{x} \sum_{i \in I} \big(\pi_i - c_{ij}\big) x_i \qquad \text{s.t.} \quad \sum_{i \in I} a_i x_i \le b, \quad x \in \{0,1\}^n,$$

with profit $\pi_i - c_{ij}$ and weight a_i for item i. Items of negative profit are simply never selected. Under the Part 3 structure all m knapsacks share the same weights a and capacity b, differing only in profits.

How the answer is used.

- If $z_j < 0$ for some j, the optimal pattern $x^\star$ attaining it is a column of negative reduced cost. Add $(j, x^\star)$ to K', re-solve the RMP, and repeat. Adding the most negative column for every agent at once is usual, since the m knapsacks are solved anyway.
- If $z_j \ge 0$ for *every* $j \in J$, then no column of the full master has negative reduced cost. The current RMP basis is therefore optimal for the *full* master LP even though most of its columns were never generated, and column generation stops.

A bound along the way. The pricing values also certify progress before termination. Since each convexity row can be used once,

$$z_{\text{master LP}} \;\ge\; z_{\text{RMP}} + \sum_{j \in J} \min\{0,\, z_j\},$$

a valid lower bound available at every iteration, which lets the loop be stopped early at a controlled optimality gap.

Remark 3. The master LP bound is at least as strong as the LP relaxation of (1)–(3), because the master optimizes over $\text{conv}(X_j)$ rather than over the box relaxation of X_j, and $\text{conv}(X_j)$ is generally strictly smaller. On the verification instance in `p2_colgen.py` (7 jobs, 3 agents, identical a_i and b) the original LP relaxation gives 57 while the master LP gives 61, which equals the integer optimum. Column generation starting from artificials alone reproduces that 61 using 35 of the 165 columns the full master would contain.

7. Initialization

Why arbitrary patterns fail. The RMP must be feasible before it can produce the duals (π, μ) that pricing needs. Feasibility requires choosing, for each agent, one pattern (convexity rows) such that the chosen patterns cover *every* job *exactly* once (covering rows). In other words, the initial columns must contain a partition of I into m capacity-feasible subsets — a feasible GAP solution in itself. A randomly chosen set of feasible patterns is essentially never such a partition: patterns may overlap on a job or leave a job uncovered, and either makes the RMP infeasible. There are then no duals, and the algorithm cannot even start.

Strategy (a): artificial columns, the robust default. Add one artificial variable $s_i \geq 0$ per covering row, priced at a large M:

$$\min \sum_j \sum_{k \in K'_j} C_{jk}\lambda_{jk} + M \sum_{i \in I} s_i \quad \text{s.t.} \quad \sum_j \sum_{k \in K'_j} x_i^k \lambda_{jk} + s_i = 1 \ \ \forall i,$$

with the convexity rows unchanged. Initialize $K'_j = \{x = 0\}$, the empty pattern, which lies in X_j for every j since $\sum_i a_i \cdot 0 = 0 \leq b$. Then $\lambda_{j,\emptyset} = 1$ and $s = \mathbf{1}$ is feasible, so the RMP has a solution and well-defined duals from the first iteration. The artificials are priced out as real columns arrive. This also yields an **infeasibility certificate**: if at termination some $s_i > 0$, no assignment of job i is possible and the GAP itself is infeasible.

Strategy (b): a construction heuristic, for speed. Run a greedy pass — order jobs by decreasing a_i and place each on the cheapest agent with enough residual capacity, a first-fit-decreasing rule — and use the resulting m patterns as the initial columns. When it succeeds, the RMP starts feasible with no artificials and with duals already close to optimal, which markedly shortens the loop. When it fails, fall back to (a). Combining the two is the usual practice: heuristic columns *and* artificials, so that feasibility never depends on the heuristic succeeding.

Strategy (c): a two-phase master. Solve an auxiliary master minimizing $\sum_i s_i$ with the same columns and the same pricing. If its optimum is 0 the artificials have left the basis and the resulting basis starts phase two; if it is positive the problem is infeasible. This avoids choosing M, which is the one weakness of (a): too small an M distorts the objective, too large an M degrades the conditioning of the master.

What does not work. Seeding with the n singleton patterns $\{i\}$ is tempting but insufficient on its own: each agent contributes exactly one pattern, so m singletons cover at most m jobs, leaving the covering rows unsatisfiable whenever $n > m$ — the normal case.

`p1_simplex.py` implements the simplex method and reproduces the hand computation of Part 1; `p1_benchmark.py` generates the instances of Part 3 and runs the comparison of Part 4; `p1_limitations.py` demonstrates the failures discussed in Part 5. Problem 2 requires no implementation; `p2_colgen.py` is an extra check that enumerates the full master, runs column generation from artificials, and confirms the two agree and that the master bound dominates the LP relaxation.